

Anexos técnicos — riesgos de HTML/JS en recursos educativos

Ernesto Serrano Collado · info@ernesto.es

2026-06-14

Índice

A. Metodología	1
B. La sonda (<code>probe.js</code>) — salida censurada	1
C. Resultados por plataforma	2
D. Resultados por navegador	3
E. Comportamiento del navegador (referencia)	3
F. Compatibilidad	3
G. Mitigaciones emparejadas (riesgo → mitigación → limitación)	3
H. Limitaciones metodológicas	4
I. Trabajo futuro	4
J. Notas de seguridad de esta investigación (checklist cumplido)	4
Anexo — Confirmación en ejecución en una instalación de Moodle en línea (2026-06-13)	4

Material de soporte del artículo. Para la matriz completa por `archivo:línea`, ver `matriz-seguridad.md`. Aquí: metodología, PoC censuradas, resultados por plataforma/navegador, comportamiento del navegador, limitaciones y trabajo futuro.

A. Metodología

1. **Verificación de código (estática).** Barrido de solo lectura de los 10 repositorios contra su HEAD actual (proceso paralelo de 10 agentes). Cada afirmación se ancla a `archivo:línea + sha`. Las versiones y SHAs analizados se listan en la sección 3.1 del artículo (Metodología).
2. **Prueba en ejecución (dinámica).** Entornos Docker locales y desechables. Sonda inyectada en el iframe del contenido y lectura de booleanos. Navegador: **dos motores** (Chromium y **Firefox/Gecko**) vía Playwright (`evidencias/resultados-firefox*.json`).
3. **Separación hechos / inferencias.** [hecho] = verificado en código o prueba; [inferencia] = deducción del comportamiento estándar del navegador.

B. La sonda (`probe.js`) — salida censurada

15 comprobaciones, salida solo booleana + nombre de error. Reglas duras: sin red, sin POST, sin lectura de valores reales, sin mutadores SCORM, popup abierto y cerrado al instante.

Fragmentos representativos (el código completo está en `poc/probe.js`):

```
// Solo se registra el NOMBRE del error, nunca el mensaje (podría llevar valores).
function errName(e){ return e && (e.name || 'Error'); }
```

```
// Cookie del padre: solo se comprueba SI es legible; el valor nunca se guarda ni se imprime.
try {
  var c = window.parent.document.cookie;      // lanza SecurityError si cross-origin/opaco
  R.canReadParentCookie = (typeof c === 'string');
} catch (e) { R.errors.cookie = errName(e); }
```

```
R.parentCookieValue = 'REDACTED';
```

// SCORM: se DETECTA el objeto API recorriendo parents; NUNCA se invoca un método.

```
while (win && tries < 20) {
  if (win.API_1484_11) { api = win.API_1484_11; break; }
  if (win.API) { api = win.API; break; }
  win = win.parent; tries++;
}
```

```
R.canCallScormApi = !!api; // reachable; deliberadamente NO se llama
```

Las cuatro PoC se generan de forma reproducible con poc/build.sh (ver poc/README.md).

C. Resultados por plataforma

Plataforma	Modo	iframe sandbox (runtime/código)	same-origin	Resultado
mod_exelearning	en ejecución	allow-scripts allow-same-origin allow-popups allow-forms allow-popups-to-escape-sandbox	sí	acceso al padre/cookie/sesskey/forms true; canCallScormApi:true (1.2)
mod_scorm (core)	en ejecución	sin sandbox (scorm_object)	sí	acceso total same-origin; canReachScormApi:true (1.2)
H5P · parámetros (mod_h5pactivity)	en ejecución (control negativo)	iframe externo sin sandbox; interno about:blank (hereda origen)	sí	frame interno same-origin (canReadTopDocument:true); los parámetros de contenido no ejecutan (filtrados)
H5P · librería (preloadedJs)	código + paquete válido + procedimiento manual	(igual: about:blank, hereda origen)	sí	el preloadedJs de una librería se ejecuta same-origin sin sandbox; la barrera es la capacidad h5p:updatelibraries (gestión). Verificado sobre el código y con PoC validada estructuralmente; ejecución <i>end-to-end</i> por procedimiento manual reproducible
mod_page	en ejecución	n/a (no iframe)	sí	<script> y EJECUTADOS ; sin saneo server-side (noclean); restringido por capacidad
Omeka S (vista pública)	en ejecución	allow-same-origin allow-scripts allow-popups allow-popups-to-escape-sandbox	sí	canAccessParent/Document/Cookie:true;
wp-exelearning	en ejecución	allow-scripts allow-same-origin allow-popups	sí	canCallScormApi:false; eval no bloqueado canAccessParent/Document:true; localiza /wp-admin/;
mod_exeweb / mod_exescorm	código	sin sandbox	sí	eval no bloqueado esperado: acceso total same-origin

“en ejecución” = ejecutado en el navegador en esta sesión (Moodle 5.0.7 local). “código” = verificado en fuente; el resultado de la sonda se deduce del **sandbox/origen**.

Resultado en ejecución Omeka (crudo)

```
{
  "platform": "omeka-s public item view",
  "iframeSandbox": "allow-same-origin allow-scripts allow-popups allow-popups-to-escape-sandbox",
  "injectedProbe": {
    "isOpaqueOrigin": false,
    "canAccessParent": true,
    "canReadParentDocument": true,
    "canReadParentCookie": true,
    "canFindCsrf": false,
    "canFindEditForms": false,
    "canUseLocalStorage": true,
    "canUsePostMessage": true,
    "canCallScormApi": false
  }
}
```

(También en evidencias/resultados-vivos.json; captura en evidencias/01-omeka-item-view.png.)

D. Resultados por navegador

- **Chrome/Chromium** (probado): el iframe con `allow-scripts + allow-same-origin` sobre contenido del propio origen permite el acceso al padre y muestra el aviso conocido de que puede “escapar” del sandbox. `srcdoc + sandbox` sin `allow-same-origin` → origen opaco, acceso al padre bloqueado (`SecurityError`).
- **Firefox/Gecko (Playwright)** (probado) — *[hecho]*: replicamos la comprobación con Playwright (evidencias/firefox-isolation-test.cjs, firefox-moodle-test.cjs). El resultado es **idéntico al de Chromium**: el iframe con `allow-same-origin` lee el padre; sin él, el documento es **opaco** y el acceso lanza `SecurityError`. Las incrustaciones reales en modo seguro de `mod_exelearning` (servido por `tokenpluginfile`), `wp-exelearning` y `omeka-s-exelearning` resultan **opacas** también en Firefox (contentWindow lanza `SecurityError` en las tres) — UA ...rv:146.0 Gecko/20100101 Firefox/146.0; datos en `resultados-firefox.json` y `resultados-firefox-moodle.json`.
- **Safari / WebKit** (no probado) — *[inferencia]*: el modelo de orígenes y el atributo `sandbox` están estandarizados (HTML Living Standard), por lo que se espera un comportamiento de aislamiento equivalente; diferencias menores posibles en `SameSite` por defecto y en políticas de cookies de terceros. Marcado como trabajo futuro.

E. Comportamiento del navegador (referencia)

- **Cookies HttpOnly** — no aparecen en `document.cookie`; un script (aunque sea same-origin) no las lee. Reducen el impacto de `canReadParentCookie: true`, pero no protegen tokens que estén en el DOM.
- **SameSite** — `Strict/Lax` limitan el envío de cookies en peticiones cross-site; `None` (o ausencia, según navegador) las envía. No sustituyen al aislamiento por origen.
- **sesskey / nonce / CSRF token** — su seguridad depende de la **validación server-side por acción**, no de ocultarlos. Un script same-origin que los lee del DOM no logra nada si el servidor revalida capacidad + token.
- **sandbox sin allow-same-origin** → **origen opaco**: `document.cookie` vacío/inaccesible útil, `localStorage` lanza o queda aislado por origen opaco, sin acceso a `parent.document`.
- **sandbox con allow-same-origin + allow-scripts** → el aislamiento es nominal: el contenido del propio origen puede manipular su relación con el padre.
- **postMessage mal validado** — aceptar mensajes sin comprobar `event.origin` y `event.source` convierte el puente en un canal de control para cualquier ventana.
- **localStorage en origen opaco** — particionado/!inaccesible; no se comparte con el origen real de la plataforma.

F. Compatibilidad

- **SCORM** — asume mismo origen y descubrimiento de `window.API` por recorrido de `parent`. Aislar por origen exige un puente `postMessage` que emule el contrato **síncrono** de pipwerks (`cmi{}` local en el hijo, `flush` en `beforeunload/LMSFinish`).
- **H5P** — ya usa `postMessage` (aunque sin validar `event.origin`); su seguridad no depende del origen sino del contenido curado + `filterParameters`.

G. Mitigaciones emparejadas (riesgo → mitigación → limitación)

Riesgo	Mitigación	No resuelve
Lectura de cookies de sesión	<code>HttpOnly + SameSite=Strict</code>	XSS same-origin; tokens en el DOM
Acceso al <code>parent</code> / mismo origen	Origen opaco / <code>srcdoc</code> / subdominio	Rompe SCORM directo; exige puente <code>postMessage</code>
Formularios con <code>sesskey/nonce</code>	Validación server-side por acción + capacidades	Nada si el servidor confía en el cliente
<code>postMessage</code> no validado JS arbitrario	<code>Allowlist + event.origin</code> y <code>event.source</code> <code>CSP + sandbox</code> sin <code>allow-same-origin</code>	<code>Allowlist</code> mal mantenida SCORM/H5P legítimos exigen arquitectura de puente
Navegación superior / popups	Omitir <code>allow-top-navigation</code> / <code>allow-popups-to-escape-sandbox</code>	Puede degradar UX de contenido legítimo

Principio rector: la mitigación efectiva vive en el **servidor** y en las **cabeceras**.

H. Limitaciones metodológicas

- Prueba en ejecución sobre `mod_exelearning`, `mod_scorum`, `mod_h5pactivity`, `mod_page` (Moodle 5.0.7 local), `wp-exelearning` (WordPress wp-env) y Omeka S. Solo `mod_exeweb/mod_exescorm` quedan verificados en código; su resultado de sonda es equivalente a casos ya ejecutados (same-origin, sin sandbox).
- En `wp-exelearning` el `evil.elpx` se publicó vía `wp media import` + el comando del plugin `wp exelearning reprocess` (extracción) y un bloque `exelearning/elp-upload server-rendered`; el fichero temporal se eliminó tras la prueba.
- Las pruebas Moodle se hicieron como **administrador**; `canFindSesskey/canFindCourseEditForms` reflejan ese rol. Con rol estudiante, `canFindCourseEditForms` sería `false` (el resto del acceso same-origin se mantiene).
- **Gotcha de entorno observado:** el `version sentinel` 9999999999 del plugin hace que Moodle no detecte cambio de versión y **no actualice el esquema** de `mdl_exelearning`; con volúmenes sembrados por una versión previa, una consulta a la columna nueva (`completionstatusrequired`) lanza `dml_read_exception` al reconstruir la caché de un curso con actividades eXeLearning. Las pruebas de `mod_page` se hicieron por ello en un curso limpio sin actividades eXeLearning.
- Dos motores probados (Chromium y **Firefox/Gecko**, vía Playwright; `evidencias/resultados-firefox*.json`). Safari/WebKit: inferencia por estándar (trabajo futuro).
- Versiones concretas (ver SHAs). El comportamiento puede cambiar entre versiones; toda cita es reproducible contra el `sha` indicado.
- La PoC mide **capacidad**, no **explotación**: detecta que el acceso es posible; no demuestra un exploit funcional (por diseño ético).

I. Trabajo futuro

- Ampliar la automatización *end-to-end* de las pruebas en ejecución ya documentadas, en especial el seguimiento SCORM y los flujos H5P / `wp-exelearning`.
- Repetir en **Safari/WebKit**; Firefox/Gecko ya está verificado vía Playwright.
- **Automatizar la verificación end-to-end del vector de librería H5P:** el selector de ficheros de Moodle 5 no se automatiza de forma fiable en *headless*, por lo que la ejecución de `preloadedJs` se confirma hoy con un procedimiento manual reproducible (`evidencias/resultados-h5p-library.json`).
- Ampliar la cobertura *end-to-end* del puente SCORM del modo seguro (origen opaco + `postMessage`) con una suite completa de seguimiento (el seguimiento básico ya está validado en prototipo).
- Medir el impacto del *toggle* CSP estricto-con-excepción para contenido externo (MathJax, YouTube).

J. Notas de seguridad de esta investigación (checklist cumplido)

- ☒ Sin cookies/tokens/`sesskey` reales en logs ni en el artículo.
- ☒ Sin endpoints externos ni código de exfiltración.
- ☒ **La sonda distribuida (`probe.js`) no hace POST ni red** (solo detecta capacidades). Las confirmaciones de impacto (anexo siguiente) usaron POST reales **autorizados y reversibles** sobre cuentas propias/de laboratorio, nunca destructivos ni en producción.
- ☒ Entornos locales y desechables; nada de producción.
- ☒ PoC didácticas e inocuas (solo booleanos + error censurado).
- ☒ Repos de plugin no modificados ni commiteados.

Anexo — Confirmación en ejecución en una instalación de Moodle en línea (2026-06-13)

Prueba **autorizada por la persona propietaria** de la cuenta sobre su **propio perfil** en una instalación de Moodle en línea de pruebas (HTTPS; host y cuenta anonimizados). El recurso no confiable se empaquetó como **SCORM** y se abrió con una **cuenta de prueba con rol de profesorado** en el curso. No se atacó a terceros ni se escaló privilegio; el único cambio fue la foto de la propia cuenta (reversible).

Aislamiento observado

El SCORM se ejecuta **same-origin** (origen no opaco): el contenido leyó y modificó `window.parent.document` (intercambio de avatar en el DOM) y localizó el `sesskey` del padre. Confirma en un servidor real la brecha descrita para SCORM sin sandbox.

Frontera de capacidades (el modelo de seguridad aguanta para terceros)

- `core_user_update_users` (vía `/lib/ajax/service.php, ajax=true`) → **rechazado**: un profesor no tiene `moodle/user:update`.
- `/user/editadvanced.php?id=5` (edición de **admin**) → **sin formulario**: un profesor no puede abrir la edición avanzada.

Es decir: un usuario no privilegiado **no puede mutar a otros usuarios**.

Lo que sí funciona: autoedición del propio perfil (nombre + foto)

Cualquier usuario autenticado puede cambiar **su propio** nombre y foto. Se confirmó la **autoedición persistente del propio perfil** mediante el **reenvío del formulario legítimo de edición de usuario** de Moodle: un script *same-origin* obtiene el `sesskey` del DOM, sube una imagen al área *draft* del propio perfil y reenvía el `FormData` del formulario real —que ya incluye el `sesskey` y los campos ocultos—, sobrescribiendo nombre y foto. (*Evidencia conceptual: no se listan aquí los endpoints ni los nombres de campo concretos; la técnica es un reenvío del propio formulario, sin escalada de privilegio.*) La operación se realizó sobre una **cuenta propia de laboratorio** y se **revirtió**.

Conclusión para el artículo

El riesgo **escala con el privilegio de quien abre el recurso**: - **Alumno/profesor**: no puede tocar a terceros, pero **sí** puede ser inducido a cambiar su **propio** nombre y foto de forma **silenciosa** (auto-suplantación / desfiguración del propio perfil), sin interacción. - **Administrador**: la misma técnica, vía `core_user_update_users` o `editadvanced.php`, permitiría **mutar a otros usuarios**.

Mitigación verificada: el **modo iframe seguro** (origen opaco) sirve el recurso con **origen opaco** → leer `window.parent/M.cfg.sesskey` lanza `SecurityError`, lo que **corta toda la cadena** (el contenido ya no comparte origen ni sesión con el LMS).

Evidencia estructurada: `evidencias/resultados-moodle-online.json`. Flujo capturado con las Dev-Tools del navegador (network): subida al *draft* (200) → reenvío del formulario de usuario (200) → redirección de éxito al perfil.